DOCKER INTERVIEW QUESTIONS & ANSWERS Docker Basics + Intermediate level

1. What is Docker?
Definition: Docker is a containerization platform used to package, distribute, and run applications in isolated containers.
Simple meaning:
Docker allows us to package an application along with its required dependencies, libraries, configuration, and runtime environment into a container.
This helps the application run consistently across different environments such as development, testing, and production.
Without Docker, an application may work on one system but behave differently on another system because of differences in operating system, software versions, dependencies, or configuration.
With Docker, the application and its required environment can be packaged together so that the same packaged application can be used across different environments.
Docker uses containers to isolate applications from each other while allowing them to share the underlying host operating system kernel.
A container is lightweight because it does not normally require a complete guest operating system like a traditional virtual machine.
Docker mainly helps with application packaging, portability, consistency, isolation, deployment, and environment management.
Why do we use Docker?
We use Docker to package applications and their dependencies into containers so that they can run consistently across different environments.
Docker helps us:

- **Package applications with their dependencies.**
- **Run applications in isolated containers.**
- **Maintain consistency across environments.**
- **Avoid dependency and configuration conflicts.**
- **Make application deployment easier.**
- **Improve portability between systems.**
- **Scale applications by running multiple containers.**
- **Support development, testing, CI/CD, and production deployments.**

What does Docker provide?
Docker provides different components and features used to build and run containerized applications.
Dockerfile is used to define instructions for building an image.
Docker Image is the packaged template used to create containers.
Docker Container is the isolated environment where the application runs.
Docker Network allows containers to communicate with each other and with external networks.
Docker Volume provides persistent storage for container data.
Docker Compose is used to define and manage multiple application services together.
Docker Registry is used to store and distribute Docker images.
Main purpose of Docker:
Docker helps us package an application and its required environment into a container so that the application can be built, distributed, and run consistently across different environments.

2. What is Docker Image?
Definition: A Docker Image is a packaged, read-only template that contains everything required to run an application, including application files, dependencies, libraries, configuration, and the required runtime environment.
Simple meaning:
A Docker Image is a package that contains everything an application needs to run.
Instead of installing the application, dependencies, libraries, and runtime separately on every system, we can package them into a Docker image.
The image itself is not a running application. It acts as a template from which Docker creates containers.
A Docker image is created using instructions written in a Dockerfile. During the image build process, Docker processes those instructions and creates the image.
Once the image is available, we can use the same image to create one or more containers.
One Docker image can be used to create multiple containers. Each container is a separate instance created from the same image.
Why do we use Docker Images?
We use Docker images to package an application and its required environment into a consistent unit.
Without an image, we may need to install and configure the application environment separately on every system.
With a Docker image, the same packaged application environment can be used across:

- Development
- Testing
- CI/CD
- Production

This helps reduce environment differences and makes application deployment more consistent.
Docker images can also be stored in a Docker Registry so that they can be shared and deployed on other systems.
What does a Docker Image contain?
A Docker image can contain:

- Application files
- Application dependencies
- Required libraries

- Runtime environment
- Configuration required by the application
- Filesystem layers created during the image build

The image is made up of layers, and Docker can reuse unchanged layers between builds, which can make image builds more efficient.

Example

Suppose we have a Node.js application.
We create a Dockerfile that defines the Node.js runtime, application files, dependencies, and startup configuration.
When we build it:
docker build -t myapp .
Docker creates an image named myapp.
We can then create a container from that image:
docker run -d --name mycontainer myapp
Here:

- myapp → Docker Image
- mycontainer → Container created from the image

We can also create another container from the same image:
docker run -d --name mycontainer2 myapp
Now the same myapp image is used as the template for both containers.
The image remains the packaged template, while the containers are the instances created from that image.

---

3. What is Docker Container?

Definition: A Docker Container is an isolated runtime environment created from a Docker image in which an application process runs.

Simple meaning:

A Docker Container is the actual environment where the application runs.
A Docker image is a packaged template, while a container is an instance created from that image.
When Docker creates a container, it uses the image as the base and creates a separate writable container layer on top of the image layers.
The container provides isolation for the application process and its filesystem, network, environment, and other runtime settings.
A container can be running or stopped. Even when a container is stopped, the container itself still exists until it is removed.

Why do we use Docker Containers?

We use containers to run applications in isolated and consistent environments.
Containers help us:

- Isolate applications from each other.
- Package applications with their required environment.
- Run multiple application instances on the same host.
- Start and stop applications easily.
- Deploy the same application consistently across environments.

Containers are generally lightweight compared with virtual machines because containers share the host operating system kernel rather than requiring a complete guest operating system for each container.

What can we do with a Docker Container?

Docker allows us to:

- Create containers.
- Start containers.
- Stop containers.
- Restart containers.
- Remove containers.
- View container status.
- View container logs.
- Execute commands inside running containers.

A container can be created without starting it, or created and started together.

Example

Suppose we already have an image called myapp.
We can create a container using:
docker create --name mycontainer myapp
This creates the container but does not start it.
To start the container:
docker start mycontainer
Now the application can run inside the container.
We can also create and start a new container together:
docker run -d --name mycontainer myapp
The difference is:

- docker create → Creates a container.
- docker start → Starts an existing container.
- docker run → Creates and starts a new container.

We can check running containers using:
docker ps
We can check all containers, including stopped containers, using:

docker ps -a

---

4. What is Docker Volume?

Definition: A Docker Volume is a Docker-managed storage mechanism used to persist data outside the container's writable layer.

Simple meaning:

A Docker Volume is used when we want application data to remain available even if the container is removed.
Normally, a container has its own writable layer where application changes can be stored.
But container storage is tied to the container lifecycle. If the container is removed, data stored only in that container can also be lost.
For applications such as databases, losing data when a container is recreated is not acceptable.
Docker volumes provide storage that exists separately from the container.
The volume can remain even after the container is removed, and another container can use the same volume.

Why do we use Docker Volumes?

We use Docker volumes when application data needs to persist independently of the container.
Volumes are commonly used for:

- Database data.
- Application persistent data.
- Uploaded files.
- Shared application data.
- Data that must survive container recreation.

The main advantage is that the container can be replaced while the required data remains in the volume.

What does a Docker Volume provide?

A Docker volume provides:

- Persistent storage.
- Storage independent of a specific container.
- Docker-managed storage.
- Ability to attach the same volume to another suitable container.
- Better separation between application runtime and persistent data.

Docker manages the actual location of the volume on the host.

Example

Suppose we have a MySQL container and we want its database data to persist.
We can create a volume:
docker volume create mydata
Then attach it to the MySQL container:
docker run -d --name mysql -v mydata:/var/lib/mysql mysql
Here:

- mydata → Docker Volume
- /var/lib/mysql → MySQL data directory inside the container

Now the MySQL data is stored in the volume.
If we remove the container:
docker rm -f mysql
The container is removed, but the mydata volume can remain.
We can then attach the same volume to another suitable MySQL container.
We can list volumes using:
docker volume ls

---

5. What is Docker Network?

Definition: Docker Network is a networking mechanism that allows Docker containers to communicate with each other and with external networks.

Simple meaning:

A Docker Network provides the communication path between containers and other network endpoints.
Suppose an application has a frontend, backend, and database.
The frontend needs to communicate with the backend, and the backend needs to communicate with the database.
Docker networking allows these containers to communicate while keeping their application processes isolated.
Containers connected to the same user-defined network can communicate with each other using their container or service names.

Why do we use Docker Networks?

We use Docker networks to provide controlled communication between containers.
Docker networking helps us:

- Connect multiple containers.
- Allow application-to-application communication.
- Separate different application networks.
- Provide name-based communication between containers.
- Connect containers to external networks when required.

For example, a backend container can communicate with a database container without depending on a manually assigned IP address.

What does Docker Networking provide?

Docker networking can provide:

- Container-to-container communication.
- Container-to-host/external communication.

- Network isolation.
- Name-based service discovery on user-defined networks.
- Port publishing for external access.

Different Docker network drivers provide different networking behavior.
A commonly used network for containers on a single Docker host is the bridge network.
Example
Suppose we create a user-defined network:
docker network create mynetwork
Then we create a backend container:
docker run -d --name backend --network mynetwork mybackend
And a database container:
docker run -d --name database --network mynetwork mysql
Now both containers are connected to mynetwork.
The backend can communicate with the database using the database container name:
database:3306
We can view Docker networks using:
docker network ls
We can inspect a network using:
docker network inspect mynetwork

---

6. What is Docker Compose?
Definition: Docker Compose is a tool used to define and manage multiple application services and their configurations using a YAML file.
Simple meaning:
Docker Compose allows us to define an application's multiple containers/services in one configuration file and manage them together.
Suppose our application contains:

- Frontend
- Backend
- Database

Without Compose, we may need to create and configure each container separately.
We may also need to separately configure:

- Ports
- Networks
- Environment variables
- Volumes
- Images
- Dependencies

Docker Compose allows us to define these settings together in a Compose YAML file.
Docker can then use that file to create and manage the application's services.
Why do we use Docker Compose?
We use Docker Compose to simplify the management of multi-container applications.
It helps us:

- Define multiple services in one file.
- Configure services consistently.
- Create application networks.
- Configure volumes.
- Configure environment variables.
- Start multiple services together.
- Stop and remove the application together.

Compose is commonly useful for development, testing, and other environments where multiple application services need to be managed together.
What does Docker Compose define?
A Compose file can define:

- Services
- Images
- Build configuration
- Ports
- Networks
- Volumes
- Environment variables
- Service dependencies
- Other service configuration

A commonly used Compose file name is:
compose.yaml
Example
Suppose our application has a frontend, backend, and database.
We can define these services in compose.yaml.
After the configuration is created, we can start the application using:

docker compose up -d
Docker Compose creates and starts the required services according to the Compose configuration.
To stop and remove the application resources created by Compose:
docker compose down
So instead of managing every application container separately, Compose allows us to manage the complete multi-service application from one configuration.

---

7. What is Docker Registry?
Definition: A Docker Registry is a storage and distribution system used to store and distribute Docker images.
Simple meaning:
A Docker Registry is a place where Docker images are stored so that they can be pushed to and pulled from different systems.
Suppose we build an application image on a developer machine.
That image exists locally on that machine.
If another server needs the same image, instead of rebuilding the image there, we can push the image to a registry.
The server can then pull the same image from the registry.
Why do we use Docker Registries?
We use Docker registries to:

- Store Docker images.
- Share images between systems.
- Distribute images to servers.
- Support application deployment.
- Maintain different image versions.

Registries can be public or private depending on the organization's requirements.
A private registry is commonly used when application images should only be accessible to authorized users or systems.
What does a Docker Registry provide?
A registry provides a central location for:

- Docker image repositories.
- Image tags.
- Image versions.
- Image distribution.
- Image access control in supported private registries.

The registry stores the image so that Docker clients or deployment systems can pull it when required.
Example
Suppose we build an image:
docker build -t myapp .
We can tag it for a registry:
docker tag myapp username/myapp:latest
Then push it:
docker push username/myapp:latest
The image is now stored in the registry.
Another machine can download it using:
docker pull username/myapp:latest
It can then create a container from the image.
The basic flow is:
Build Image → Push to Registry → Pull from Registry → Run Container

---

8. What is Docker Hub?
Definition: Docker Hub is a cloud-based public Docker registry service provided by Docker that is used to store, share, and distribute container images.
Simple meaning:
Docker Hub is an online registry where Docker images can be stored and downloaded.
Developers can push their application images to Docker Hub and other systems can pull those images when required.
Docker Hub also provides repositories where images can be organized and distributed.
Why do we use Docker Hub?
We use Docker Hub to:

- Store Docker images online.
- Share images with other users.
- Download existing images.
- Distribute application images.
- Use publicly available images.
- Store private images where supported by the account and repository configuration.

Docker Hub is also commonly used to obtain existing images such as official or publisher-provided images.
What can we do with Docker Hub?
We can:

- Create repositories.
- Push images.
- Pull images.
- Tag image versions.
- Share public images.

- Manage private repositories according to the available plan and permissions.

Example
Suppose we build an application image:
docker build -t myapp .
We can tag it for a Docker Hub repository:
docker tag myapp username/myapp:latest
After logging in:
docker login
We can push the image:
docker push username/myapp:latest
Another machine can pull the image:
docker pull username/myapp:latest
The machine can then use that image to create a container.
Docker Hub therefore acts as a cloud-based registry for storing and sharing Docker images.

---

9. What is Docker Build Context?
Definition: A Docker Build Context is the set of files and directories that Docker can access during an image build.
Simple meaning:
Docker Build Context is the collection of files that Docker receives and can use while building a Docker image.
When we run a Docker build command, we specify a location as the build context. Docker uses that location as the source of files available to instructions such as COPY and ADD.
The Dockerfile is normally located inside the build context, but the Dockerfile and build context are conceptually different.
The build context can contain application source code, configuration files, dependency files, and other files required during the image build.
Docker cannot use arbitrary files outside the build context through normal COPY or ADD instructions.
A large build context can increase the amount of data Docker needs to process during the build. Therefore, unnecessary files should be excluded from the build context using .dockerignore.
Why do we use Docker Build Context?
We use the build context to define which files are available to Docker during the image-building process.
It helps us:

- Provide application files required during the build.
- Control which files Docker can access.
- Keep the build process organized.
- Avoid sending unnecessary files to the Docker builder.
- Improve build efficiency when unnecessary files are excluded.

What does Docker Build Context contain?
The build context can contain:

- Application source code.
- Dependency files.
- Configuration files.
- Static files.
- Dockerfile-related files.
- Other files required during the image build.

Files excluded through .dockerignore are not included in the build context sent to the builder.

---

10. What is Docker CMD vs ENTRYPOINT?
Definition: CMD and ENTRYPOINT are Dockerfile instructions used to define the command or executable that should run when a container starts.
Simple meaning:
CMD and ENTRYPOINT both control what runs when a container starts, but they have different purposes.
CMD provides the default command or default arguments for the container.
ENTRYPOINT defines the main executable or application that the container is intended to run.
CMD can be overridden when a command is provided during docker run.
ENTRYPOINT normally remains the main executable, while CMD can provide default arguments to it.
Why do we use CMD and ENTRYPOINT?
We use CMD and ENTRYPOINT to define the container's startup behavior.
They allow us to specify:

- Which application should start.
- What default command should run.
- What default arguments should be used.
- Which executable should act as the main process.

The choice between CMD and ENTRYPOINT depends on whether we want the container's startup command to be easily replaced or whether we want to define a fixed main executable.
What is CMD?
CMD defines the default command or default arguments for a container.
If another command is supplied when the container is started, the runtime command can replace the image's CMD.
CMD can therefore be used when we want to provide a default behavior that can be changed when required.
What is ENTRYPOINT?
ENTRYPOINT defines the main executable of the container.
It is commonly used when the container is designed to run a specific application or executable.

When CMD is used together with ENTRYPOINT, CMD can provide default arguments to the ENTRYPOINT.

Important difference

- CMD → Default command or default arguments.
- ENTRYPOINT → Main executable.

CMD is generally easier to override, while ENTRYPOINT is used when the container should have a defined primary executable.

---

11. What is Docker COPY vs ADD?

Definition: COPY and ADD are Dockerfile instructions used to copy files and directories into a Docker image.

Simple meaning:

Both COPY and ADD can bring files from the build context into the image, but ADD provides some additional functionality.
COPY is mainly intended for straightforward copying of files and directories.
ADD can also copy files and directories, and it provides additional behavior such as extracting supported local tar archives.

Why do we use COPY and ADD?

We use these instructions to include required application files and directories inside the Docker image during the build process.
They allow us to:

- Copy application source code.
- Copy configuration files.
- Copy dependency files.
- Copy directories.
- Include required files inside the image.

What is COPY?

COPY is used to copy files or directories from the build context into the image.
It provides simple and explicit file-copying behavior.
Because its behavior is straightforward, COPY is generally preferred when we only need to copy files into the image.

What is ADD?

ADD can copy files and directories into the image and also provides additional features.
One important additional behavior is that supported local tar archives can be automatically extracted when added to the image.
Because ADD has additional behavior, it should generally be used only when those additional features are actually required.

Main difference

- COPY → Simple and explicit file copying.
- ADD → File copying plus additional features such as supported local tar archive extraction.

---

12. What is Docker .dockerignore?

Definition: .dockerignore is a file used to exclude unnecessary files and directories from the Docker build context.

Simple meaning:

.dockerignore tells Docker which files should not be included in the build context.
When Docker builds an image, it sends the build context to the builder. If the project contains unnecessary files, those files may also become part of the build context unless they are excluded.
.dockerignore allows us to exclude files that are not required for building the image.

Why do we use .dockerignore?

We use .dockerignore to keep unnecessary files out of the Docker build context.
It helps us:

- Reduce the build context size.
- Improve build efficiency.
- Avoid sending unnecessary files to the builder.
- Prevent development-only files from being included.
- Reduce the chance of accidentally including sensitive or unwanted files.

What can we exclude using .dockerignore?

We can exclude files and directories such as:

- node_modules
- .git
- Log files.
- Temporary files.
- Local development files.
- Build artifacts that are not required.
- Environment or sensitive files that should not be part of the build context.

The .dockerignore file follows pattern-based rules to determine which files are excluded.

---

13. What are Docker Environment Variables?

Definition: Docker Environment Variables are configuration values provided to a container so that an application can use different settings without changing the Docker image.

Simple meaning:

Environment variables allow us to provide configuration values separately from the application image.
Instead of hard-coding environment-specific values inside the application or image, we can provide those values when the container runs.
This allows the same Docker image to be used in different environments with different configuration values.
For example, the application may require values related to database configuration, application mode, service endpoints, or other runtime settings.

Why do we use Environment Variables?

We use environment variables to separate application configuration from the application image.
They help us:

- Reuse the same image across environments.
- Change runtime configuration without rebuilding the image.
- Keep configuration separate from application code.
- Provide environment-specific values.
- Simplify application deployment.

Where can Docker Environment Variables be defined?
Environment variables can be provided through different Docker mechanisms, including:

- Dockerfile ENV instruction.
- docker run -e.
- docker run --env-file.
- Docker Compose configuration.
- Other deployment and orchestration mechanisms.

The application running inside the container can read these values from its environment.
Important point
Environment variables are useful for configuration, but sensitive values such as passwords and credentials should be handled carefully. Environment variables are not automatically a secure secret-management mechanism.

---

14. What is Docker Port Mapping?
Definition: Docker Port Mapping is the process of mapping a port on the Docker host to a port inside a container so that external clients can access an application running inside the container.
Simple meaning:
Port mapping connects a **host port** to a **container port**.
An application inside a container may listen on a particular port, but that port is not automatically exposed to external users through the Docker host.
Port publishing allows traffic arriving at a host port to be forwarded to the corresponding container port.
Why do we use Port Mapping?
We use port mapping when an application running inside a container needs to be accessed from outside the container.
It helps us:

- Make containerized applications accessible from the host.
- Allow external clients to reach container applications.
- Map different host ports to container ports.
- Control which container ports are published externally.

How does Port Mapping work?
Port mapping follows this general structure:
HOST_PORT:CONTAINER_PORT
The host port is the port exposed on the Docker host, while the container port is the port where the application is listening inside the container.
Port publishing can be configured when starting a container.

---

15. What is Docker Bind Mount?
Definition: A Docker Bind Mount is a storage mechanism that mounts a specific file or directory from the Docker host directly into a container.
Simple meaning:
A bind mount connects a specific location on the host machine to a location inside the container.
The container can access the files from the mounted host directory.
Unlike Docker-managed volumes, bind mounts depend on a specific host filesystem path.
Changes made to the mounted files on the host can be reflected inside the container, depending on the mount configuration.
Why do we use Bind Mounts?
We use bind mounts when a container needs direct access to files or directories on the host.
They are commonly useful for:

- Local application development.
- Sharing source code with containers.
- Sharing configuration files.
- Accessing host files from containers.
- Development environments where changes on the host need to be visible inside the container.

How does a Bind Mount work?
A bind mount has two important locations:

- Host path → Existing file or directory on the host.
- Container path → Location where that host path is mounted inside the container.

The host path is controlled by the user, while Docker provides the container-side mount location.
Important point
Bind Mount and Docker Volume are different.

- Bind Mount → Uses a specific host filesystem path.
- Docker Volume → Uses storage managed by Docker.

---

16. What is Docker Named Volume?

Definition: A Docker Named Volume is a Docker-managed storage location identified by a specific name and used to persist data independently of a container's lifecycle.

Simple meaning:

A named volume is persistent storage managed by Docker and identified by a name.
The volume exists separately from the container that uses it.
If the container is removed, the named volume can continue to exist and can be attached to another suitable container.
Docker manages the actual storage location of the volume on the host.

Why do we use Named Volumes?

We use named volumes when application data needs to persist beyond the lifetime of a container.
They are commonly useful for:

- Database data.
- Persistent application data.
- Uploaded files.
- Shared data that should survive container recreation.

Named volumes also provide a cleaner storage mechanism because users do not need to manage the underlying host filesystem path directly.

What does a Named Volume provide?

A named volume provides:

- Persistent storage.
- Docker-managed storage.
- A unique volume name.
- Ability to attach the volume to containers.
- Separation between container lifecycle and persistent data.

Named volumes can be listed, inspected, attached, and removed using Docker volume management operations.

---

17. What is Docker Bridge Network?

Definition: A Docker Bridge Network is a Docker network that allows containers connected to the same bridge network to communicate with each other.

Simple meaning:

A bridge network provides communication between containers running on the same Docker host.
Docker provides a default bridge network, and users can also create user-defined bridge networks.
User-defined bridge networks provide better control and provide automatic name-based communication between connected containers.

Why do we use Bridge Networks?

We use bridge networks to allow containers on the same Docker host to communicate while keeping network traffic organized and isolated from unrelated networks.
They are commonly used for:

- Backend-to-database communication.
- Communication between application containers.
- Multi-container applications.
- Isolating groups of containers.

What does a Bridge Network provide?

A bridge network can provide:

- Container-to-container communication.
- Network isolation.
- Name-based communication on user-defined bridge networks.
- Connectivity between containers and external networks through the Docker host.

Containers connected to the same user-defined bridge network can communicate using container names instead of depending on fixed IP addresses.

---

18. What is Docker Swarm?

Definition: Docker Swarm is Docker's native container orchestration technology used to manage services across a cluster of Docker hosts.

Simple meaning:

Docker Swarm allows multiple Docker hosts to work together as a cluster and provides management of containerized services across that cluster.
Instead of managing containers individually on different Docker hosts, Swarm allows us to define services and their desired state.
Swarm manages the placement of service tasks across available nodes and works to maintain the desired number of replicas.
A Swarm cluster contains manager nodes and worker nodes.
Manager nodes are responsible for maintaining cluster state and scheduling workloads, while worker nodes run assigned workloads.

Why do we use Docker Swarm?

We use Docker Swarm to manage containerized applications across multiple Docker hosts.
It provides features such as:

- Service management.
- Desired-state management.
- Service replication.
- Scheduling workloads across nodes.
- Scaling services.

- Service recovery when tasks fail.
- Cluster management.

What does Docker Swarm manage?

Docker Swarm manages:

- Nodes.
- Services.
- Tasks.
- Service replicas.
- Networks.
- Secrets.
- Configurations.

A service defines what should run, while Swarm works to maintain that desired state across the cluster.

---

19. What is Docker Service?

Definition: A Docker Service is a definition of the desired state of an application workload in a Docker Swarm cluster.

Simple meaning:

A Docker Service tells Swarm **what application should run and how many replicas should be maintained**.

A service is not simply one container.

A service represents the desired state, and Swarm creates and manages the tasks required to maintain that state.

A service can define information such as:

- Container image.
- Number of replicas.
- Published ports.
- Networks.
- Environment variables.
- Volumes.
- Other service configuration.

Why do we use Docker Services?

We use Docker services to manage application workloads in a Swarm cluster.

Services allow us to:

- Define the desired number of replicas.
- Scale applications.
- Maintain the desired state.
- Distribute workloads across nodes.
- Replace failed tasks.
- Update service configuration.

If a task fails, Swarm can create another task to return the service to its desired state.

What is the relationship between Service and Task?

A service defines the desired workload.

A task is an individual unit created by Swarm to run that workload.

Therefore:

Service → Desired State → Tasks → Running Containers

The service manages the desired state, while tasks represent the individual running workload instances.

---

20. What is Docker Stack?

Definition: Docker Stack is a mechanism used to deploy and manage a multi-service application on a Docker Swarm cluster using a Compose-format YAML configuration file.

Simple meaning:

Docker Stack allows us to define an entire multi-service application in one YAML file and deploy it to Swarm as a stack.

Instead of creating each Swarm service separately, we can define the application's services and their configurations together.

A stack can contain multiple services along with their networking, storage, replicas, ports, and other configuration.

Why do we use Docker Stack?

We use Docker Stack to deploy complete multi-service applications to a Docker Swarm cluster.

It helps us:

- Define multiple services in one YAML file.
- Deploy services together.
- Configure replicas.
- Configure networks.
- Configure volumes.
- Configure ports.
- Manage the application as one stack.
- Scale and update services through the stack configuration.

What can a Docker Stack define?

A stack configuration can define:

- Services.
- Container images.
- Replicas.
- Networks.

- Volumes.
- Ports.
- Environment variables.
- Service dependencies.
- Other supported service configuration.

1. Docker Stack uses a Compose-format YAML file as the application definition and deploys that definition as services in the Swarm
cluster.

---

21. What is Docker Healthcheck?

Definition: A Docker Healthcheck is a mechanism used to check whether the application inside a container is functioning correctly.
Simple meaning:
Healthcheck tells Docker whether the application inside the container is actually healthy or not.
A container can be in a running state even when the application inside it is not responding correctly.
Why do we use Docker Healthcheck?
We use Healthchecks to verify the actual health of the application instead of checking only whether the container process is running.
Healthchecks are useful for:

- Detecting application failures.
- Identifying unhealthy containers.
- Monitoring application availability.
- Helping Docker and other systems understand application health.

What does Docker Healthcheck define?
A Healthcheck defines:

- The command used to check application health.
- The interval between health checks.
- The timeout for a health check.
- The number of retries before the container is considered unhealthy.
- The initial startup period when required.

The HEALTHCHECK instruction is used in a Dockerfile to define a health check for the container.

---

22. What are Docker Container Restart Policies?
Definition: Docker Container Restart Policies define when Docker should automatically restart a container after it stops or exits.
Simple meaning:
Restart policy tells Docker:
"When this container stops, should Docker start it again?"
Why do we use Restart Policies?
We use restart policies to automatically recover containers from unexpected failures.
This is useful for applications that should continue running without manual intervention.
Docker provides restart policies such as:

- no → Container is not automatically restarted.
- on-failure → Container is restarted when it exits with a non-zero status.
- always → Container is automatically restarted according to Docker's restart behavior.
- unless-stopped → Container is restarted unless it was explicitly stopped.

Restart policies are commonly used for long-running applications and services.

---

23. What are Docker Logs?
Definition: Docker Logs are the standard output and error output generated by the main process running inside a container.
Simple meaning:
Docker logs allow us to see what the application inside the container is printing.
Why do we use Docker Logs?
We use Docker logs to:

- Troubleshoot application errors.
- Check application startup messages.
- Identify runtime problems.
- Monitor application output.
- Understand why a container may have failed.

Docker provides commands to view and follow container logs.
For example:
docker logs mycontainer
To continuously watch new logs:
docker logs -f mycontainer
To display only the latest logs:
docker logs --tail 100 mycontainer
Docker Compose also provides log commands for services.

---

24. What is Docker Exec?

Definition: docker exec is a Docker command used to execute a command inside an already running container.
Simple meaning:
Docker Exec allows us to enter or run commands inside a running container without creating a new container.
Why do we use Docker Exec?
We use Docker Exec for:

- Debugging containers.
- Checking files inside a container.
- Inspecting configuration.
- Checking environment variables.
- Troubleshooting application problems.

For example, if a container is already running, we can open a shell inside it:
docker exec -it mycontainer sh
We can also execute a specific command:
docker exec mycontainer ls /app

---

25. What is Docker Architecture?
Definition: Docker Architecture describes how the different Docker components communicate and work together to build, run, and manage containers.
Simple meaning:
Docker Architecture explains how a Docker command from the user reaches Docker and how Docker performs the requested operation.
The main components of Docker Architecture are:

- Docker Client → Used by the user to send Docker commands.
- Docker Daemon → Performs Docker operations and manages Docker objects.
- Docker Registry → Stores and distributes Docker images.

The Docker Client communicates with the Docker Daemon.
The Docker Daemon handles operations such as:

- Building images.
- Creating containers.
- Starting containers.
- Stopping containers.
- Managing networks.
- Managing volumes.

If a required image is not available locally, Docker can obtain it from a Docker Registry.
The basic architecture flow is:
Docker Client → Docker Daemon → Docker Objects
When an image needs to be downloaded:
Docker Client → Docker Daemon → Docker Registry → Docker Image → Container

---

26. What is Docker Engine?
Definition: Docker Engine is the core technology used to build images, run containers, and manage Docker objects.
Simple meaning:
Docker Engine is the main part of Docker that performs container-related operations.
Why do we use Docker Engine?
Docker Engine provides the functionality required to:

- Build Docker images.
- Create containers.
- Start containers.
- Stop containers.
- Remove containers.
- Manage Docker networks.
- Manage Docker volumes.
- Manage other Docker objects.

Docker Engine includes the components required for Docker operations, including the Docker daemon and the interfaces used to communicate with it.
The Docker Engine is therefore responsible for carrying out the actual Docker operations requested by the user.

---

27. What is Docker Client?
Definition: Docker Client is the command-line interface used by users to communicate with Docker.
Simple meaning:
Docker Client is the interface through which we give commands to Docker.
We use the Docker Client when we run commands such as:
docker build
docker run
docker ps
docker stop
docker pull
Why do we use Docker Client?
We use Docker Client to:

- Send commands to Docker.
- Build images.
- Create and manage containers.
- Manage networks.
- Manage volumes.
- Pull and push images.

The Docker Client sends the requested operation to the Docker Daemon.
So the Docker Client is mainly the user-facing interface, while the Docker Daemon performs the Docker management operations.

---

28. What is Docker Daemon?
Definition: Docker Daemon is a background service responsible for managing Docker objects and performing Docker operations.
Simple meaning:
Docker Daemon is the component that receives Docker requests and performs the actual work.
Why do we use Docker Daemon?
Docker Daemon manages:

- Docker images.
- Docker containers.
- Docker networks.
- Docker volumes.
- Other Docker objects.

The daemon can:

- Build images.
- Create containers.
- Start containers.
- Stop containers.
- Remove containers.
- Pull images.
- Manage networks.
- Manage volumes.

The Docker Client sends requests to the Docker Daemon, and the daemon performs those operations.

---

29. What is Docker Images vs Containers?
Definition: A Docker Image is a packaged, read-only template used to create containers, while a Docker Container is an instance created from that image.
Simple meaning:
Docker Image = Template
Docker Container = Instance
An image contains the application and the required dependencies needed to run it.
A container is created from that image and provides the runtime environment where the application runs.
Why do we use Images and Containers?
Images are used to package applications in a consistent format.
Containers are used to run those packaged applications.
One Docker Image can be used to create multiple containers.
The relationship is:
Docker Image → Container 1
Docker Image → Container 2
Docker Image → Container 3
So the same image can be used as a template for multiple container instances.

---

30. What is Docker Compose vs Docker Swarm?
Definition: Docker Compose and Docker Swarm are Docker technologies used to work with multiple services, but they are designed for different purposes.
Simple meaning:
Docker Compose → Manage multiple application services together.
Docker Swarm → Orchestrate services across multiple Docker hosts.
Docker Compose is mainly used to define and manage multi-container applications using a Compose YAML file.
Docker Swarm is Docker's native orchestration technology used to manage services across a cluster of Docker hosts.
Docker Compose focuses on application-level multi-container management.
Docker Swarm focuses on cluster-level service orchestration.
The main difference is:
Docker Compose → Multi-container application management.
Docker Swarm → Multi-host container orchestration.

---

31. What is Docker vs Virtual Machine?
Definition: Docker containers and Virtual Machines are both technologies used to isolate applications, but they use different approaches.
Simple meaning:
Virtual Machine → Virtualizes a complete operating system.
Docker Container → Isolates application processes while sharing the host operating system kernel.

A Virtual Machine normally contains:

- Guest Operating System.
- Libraries.
- Applications.
- Virtual hardware environment.

Docker containers generally contain:

- Application.
- Application dependencies.
- Required user-space files.

Containers share the host operating system kernel instead of running a complete guest operating system for every container.
Because containers do not require a separate full guest OS for each application, they are generally lighter and can start faster than traditional VMs.
Virtual Machines provide isolation at the hardware/OS virtualization level.
Containers provide process-level isolation and are commonly used for application packaging and deployment.
The basic difference is:
Virtual Machine → Guest OS → Application
Docker Container → Shared Host Kernel → Application

---

32. What is Docker Container Lifecycle?

Definition: Docker Container Lifecycle describes the different stages and operations a container goes through from creation until removal.
Simple meaning:
Container lifecycle explains what happens to a container from the time it is created until it is finally removed.
A container can go through states such as:

- Created
- Running
- Stopped
- Restarted
- Removed

A container can be created without starting it.
A stopped container can be started again.
A running container can be stopped.
A stopped container can eventually be removed.
The important Docker commands are:
docker create
Creates a container but does not start it.
docker start
Starts an existing stopped container.
docker stop
Stops a running container.
docker rm
Removes a container.
docker run
Creates and starts a new container.
The basic lifecycle is:
Docker Image → Create → Start → Running → Stop → Start Again → Remove
The Docker Image remains available as the template from which new containers can be created.

1. ---

33. What is Docker Tag?

Definition: A Docker Tag is a label used to identify a specific version or variant of a Docker image.
Simple meaning:
A Docker Tag helps us identify which image version we want to use.
The general image format is:
IMAGE_NAME:TAG
For example, an image can have tags such as:

- latest
- v1
- v2
- 1.0
- 1.1

Why do we use Docker Tags?
We use tags to:

- Identify image versions.
- Maintain multiple versions of an image.
- Select a specific image version.
- Push different image versions to a registry.
- Avoid depending only on the default tag.

If no tag is specified, Docker commonly uses latest as the default tag.

34. What is Docker Pull?

Definition: docker pull is a Docker command used to download an image from a Docker Registry to the local Docker environment.

Simple meaning:

Docker Pull means downloading an image from a registry.

Why do we use Docker Pull?

We use docker pull when:

- An image is required locally.
- We want to download an updated image.
- We want to get an image from Docker Hub or another registry.
- We want to prepare an image before creating a container.

The command format is:

docker pull IMAGE_NAME

Docker downloads the required image layers from the registry and stores the image locally.

After the image is available locally, it can be used to create containers.

---

35. What is Docker Push?

Definition: docker push is a Docker command used to upload a Docker image from the local environment to a Docker Registry.

Simple meaning:

Docker Push means uploading an image to a registry.

Why do we use Docker Push?

We use docker push to:

- Store application images in a registry.
- Share images with other systems.
- Make images available for deployment.
- Transfer images from development environments to production environments.

Before pushing, the image normally needs to have a repository name and tag associated with the target registry.

The general command is:

docker push IMAGE_NAME:TAG

After the push is completed, other systems can pull the image from the registry.

---

36. What is Docker Tag vs Image Digest?

Definition: A Docker Tag is a human-readable label for an image, while an Image Digest is a content-based identifier that uniquely identifies a specific image content.

Simple meaning:

Tag → Human-friendly image label.

Digest → Content-based image identifier.

Tags can be changed to point to different image versions.

For example, a tag such as latest may point to one image today and a different image after a new image is pushed.

A digest is based on the image content and identifies a specific image.

Why do we use Image Digests?

Digests are useful when we need:

- Exact image identification.
- Immutable image references.
- Reproducible deployments.
- Greater confidence that the same image content is being used.

So:

Tag → Easy to read and commonly used.

Digest → Exact content-based identification.

---

37. What is Docker Image Layer?

Definition: A Docker Image Layer is an individual filesystem layer that forms part of a Docker image.

Simple meaning:

A Docker image is built from multiple layers, and each layer represents changes made during the image build process.

Dockerfile instructions such as RUN, COPY, and ADD can create filesystem changes that become part of image layers.

Why do we use Image Layers?

Image layers provide:

- Reusability.
- Efficient storage.
- Faster image builds.
- Faster image transfers.
- Build caching.

When multiple images use the same base layers, Docker can reuse those layers instead of storing duplicate copies.

This helps reduce storage and transfer requirements.

The basic structure is:

Base Layer → Application Dependencies → Application Files → Final Image

---

38. What is Docker Image Cache?

Definition: Docker Image Cache is the mechanism Docker uses to reuse previously built image layers during a Docker image build.

Simple meaning:
Docker Cache allows Docker to reuse unchanged build steps instead of performing them again.
Why do we use Docker Build Cache?
We use the cache to:

- Reduce build time.
- Avoid repeating unchanged operations.
- Improve development speed.
- Make CI/CD builds more efficient.

When Docker builds an image, it checks whether a previous layer can be reused.
If the instruction and relevant inputs have not changed, Docker can reuse the existing cached layer.
If a layer changes, later dependent layers may also need to be rebuilt.
So Docker Build Cache helps avoid unnecessary work during image creation.

---

39. What is Docker EXPOSE?
Definition: EXPOSE is a Dockerfile instruction used to document the network port on which an application inside the container is expected to listen.
Simple meaning:
EXPOSE tells Docker and users which container port the application is expected to use.
For example:
EXPOSE 8080
This indicates that the application is expected to listen on port 8080 inside the container.
Why do we use EXPOSE?
We use EXPOSE to:

- Document the application's listening port.
- Make the Dockerfile easier to understand.
- Indicate the intended container network port.

Important point:
EXPOSE does **not** publish the port to the host by itself.
Host-to-container access normally requires port publishing using Docker's -p option.

---

40. What is Dockerfile ARG?
Definition: ARG is a Dockerfile instruction used to define a build-time variable that can be supplied during the image build process.
Simple meaning:
ARG allows us to pass values into the Docker image build process.
An ARG value exists during image build and is primarily intended for configuring the build process.
Why do we use ARG?
We use ARG to:

- Pass build-time configuration.
- Select versions during builds.
- Customize image creation.
- Avoid hard-coding certain build parameters.

The general structure is:
ARG VERSION=20
The value can also be provided during the build process.
Important point:
ARG is mainly for build-time configuration. It should not be treated as a secure mechanism for storing secrets because build arguments can be exposed through image build metadata or history depending on how they are used.

---

41. What is Dockerfile ENV?
Definition: ENV is a Dockerfile instruction used to define environment variables that are available in the image and containers created from it.
Simple meaning:
ENV defines configuration values that can be available to the application when the container runs.
Why do we use ENV?
We use ENV to:

- Define application configuration.
- Set environment-related values.
- Provide default configuration.
- Make configuration available to processes inside the container.

For example:
ENV APP_ENV=production
The environment variable can then be available inside the container.
ENV values can also be overridden when a container is started using appropriate Docker configuration.
Important point:
Sensitive information such as passwords and private credentials should not normally be stored directly in a Dockerfile using ENV.

---

42. What is Dockerfile WORKDIR?
Definition: WORKDIR is a Dockerfile instruction used to set the working directory for subsequent Dockerfile instructions and for the container's default working directory.

Simple meaning:
WORKDIR tells Docker:
“From this point, use this directory as the current working directory.”
For example:
WORKDIR /app
After setting the working directory, instructions such as COPY, RUN, and the default container command can operate relative to that directory where applicable.
Why do we use WORKDIR?
We use WORKDIR to:

- Organize application files.
- Define a consistent working location.
- Avoid repeatedly using absolute paths.
- Set the default working directory for the application.

It is generally preferred over manually using repeated cd commands in Dockerfile instructions.

---

43. What is Dockerfile USER?
Definition: USER is a Dockerfile instruction used to specify the user or group under which subsequent commands and the container's main process should run.
Simple meaning:
USER tells Docker which user should run the application inside the container.
By default, some images may run processes as the root user.
Running applications as a non-root user can reduce the impact of certain security vulnerabilities.
Why do we use USER?
We use USER to:

- Run applications with a non-root identity.
- Improve container security.
- Control file ownership and permissions.
- Reduce unnecessary privileges.

The Dockerfile can specify a user or group using the USER instruction.
For production applications, running with only the permissions required by the application is generally considered a good security practice.

---

44. What is Dockerfile RUN?
Definition: RUN is a Dockerfile instruction used to execute commands during the Docker image build process.
Simple meaning:
RUN performs an operation while Docker is building the image.
Common uses include:

- Installing dependencies.
- Installing packages.
- Creating directories.
- Building application files.
- Performing required setup operations.

For example:
RUN npm install
This command runs during the image build process.
The result of the command becomes part of the image filesystem through the resulting build layer.
Important point:
RUN happens during **image build**.
It is different from CMD and ENTRYPOINT, which define what happens when a container starts.

---

45. What is Dockerfile FROM?
Definition: FROM is a Dockerfile instruction used to specify the base image from which a new Docker image is built.
Simple meaning:
FROM tells Docker:
“Start building my image from this existing image.”
For example:
FROM node:20
This means the new image starts from the specified Node.js image.
Why do we use FROM?
We use FROM to:

- Select the required base environment.
- Provide the required operating-system/user-space files.
- Provide a runtime such as Node.js, Python, Java, or Nginx.
- Build our application image on top of an existing image.

FROM is normally the first major instruction in a Dockerfile.
The basic relationship is:
Base Image → Dockerfile Instructions → New Docker Image

1.

---

46. What is Dockerfile LABEL?

Definition: LABEL is a Dockerfile instruction used to add metadata to a Docker image.
Simple meaning:
LABEL adds information about the image in the form of key-value pairs.
Labels can contain information such as:

- Application name.
- Version.
- Maintainer information.
- Organization.
- Description.
- Build information.

Why do we use LABEL?
We use LABEL to:

- Add metadata to images.
- Identify images easily.
- Organize and manage images.
- Provide useful information about an image.

The labels become part of the image metadata and can be viewed when inspecting the image.

---

47. What is Dockerfile SHELL?
Definition: SHELL is a Dockerfile instruction used to specify the default shell used by shell-form commands in subsequent Dockerfile instructions.
Simple meaning:
SHELL tells Docker which shell should be used when executing shell-form commands.
The default shell can differ depending on the base image and operating system.
Why do we use SHELL?
We use SHELL to:

- Specify a particular shell.
- Change shell behavior during image builds.
- Use shell-specific features.
- Control how shell-form commands are executed.

This is mainly useful when the application requires a specific shell environment or shell behavior.

---

48. What is Dockerfile ONBUILD?
Definition: ONBUILD is a Dockerfile instruction that adds a trigger instruction to an image, which is executed when that image is later used as a base image for another Dockerfile.
Simple meaning:
ONBUILD means:
“Do this instruction when another image is built using my image as its base.”
Why do we use ONBUILD?
We use ONBUILD when creating reusable base images that should automatically perform certain build steps in child images.
It is useful for:

- Creating reusable base images.
- Defining automatic build behavior.
- Creating framework-specific base images.
- Sharing common build instructions.

The ONBUILD instruction itself does not execute the triggered instruction while building the current image.
The trigger is stored in the image and executes when another Dockerfile uses that image with FROM.

---

49. What is Dockerfile VOLUME?
Definition: VOLUME is a Dockerfile instruction used to declare a mount point for persistent or externally managed container data.
Simple meaning:
VOLUME tells Docker that a particular directory is intended to hold data that should be stored outside the container's writable layer.
Why do we use VOLUME?
We use VOLUME to:

- Identify directories that contain persistent data.
- Separate application data from the container's writable layer.
- Allow data to be stored using Docker-managed storage.
- Prevent important application data from depending only on the container filesystem.

A volume is useful for data that needs to survive container recreation.
The important idea is:
Container → Volume → Persistent Data

---

50. What is Dockerfile STOPSIGNAL?
Definition: STOPSIGNAL is a Dockerfile instruction used to specify the system signal that Docker should send to the container's main process when the container is stopped.
Simple meaning:
STOPSIGNAL tells Docker which signal should be used to request that the application's main process shut down.
Why do we use STOPSIGNAL?
We use STOPSIGNAL when an application requires a specific signal for graceful shutdown.

It can be useful for:

- Graceful application shutdown.
- Applications that handle specific signals.
- Controlling how the main process receives termination requests.
- Ensuring the application gets the expected shutdown signal.

By default, Docker uses its standard stop signal behavior, but STOPSIGNAL allows the Dockerfile to specify a different signal when required.

1.

51. What is Dockerfile HEALTHCHECK?

Definition: HEALTHCHECK is a Dockerfile instruction used to define a command that Docker can run to determine whether the application inside a container is healthy.
Simple meaning:
HEALTHCHECK tells Docker how to check whether the application is working properly.
Why do we use HEALTHCHECK?
We use HEALTHCHECK to:

- Check application health.
- Detect unhealthy containers.
- Distinguish between a running container and a healthy application.
- Provide health status information to Docker.

A healthcheck can define:

- The command used for the check.
- The interval between checks.
- The timeout.
- The number of retries.
- The startup period.

---

52. What is Docker Container Name?
Definition: A Docker Container Name is a user-defined name assigned to a Docker container so that the container can be easily identified and managed.
Simple meaning:
Container name is the easy-to-remember name given to a container.
Why do we use Container Names?
We use container names to:

- Identify containers easily.
- Manage containers using readable names.
- Execute Docker commands against specific containers.
- Avoid depending only on automatically generated container names.

A container name must be unique among containers on the same Docker host.
Docker commands can use the container name to perform operations such as:

- Start.
- Stop.
- Remove.
- Inspect.
- View logs.
- Execute commands.

---

53. What is Docker Container ID?
Definition: A Docker Container ID is a unique identifier assigned by Docker to a container.
Simple meaning:
Container ID is the unique identity of a Docker container.
Every container receives an ID when it is created.
Why do we use Container IDs?
We use Container IDs to:

- Identify a specific container.
- Manage containers.
- Inspect container information.
- Execute Docker operations against a specific container.

Docker commands can usually accept either:

- Container name
- Container ID

The Container ID is automatically generated by Docker, while the container name can be chosen by the user.

---

54. What is Docker Detached Mode?
Definition: Detached mode is a Docker execution mode in which a container runs in the background and the terminal is returned to the user.
Simple meaning:
Detached mode means the container runs in the background instead of occupying the current terminal.

We use the -d option with docker run to start a container in detached mode.
Why do we use Detached Mode?
We use detached mode for:

- Background services.
- Web applications.
- APIs.
- Databases.
- Long-running containers.

The terminal remains available after the container starts.
The container continues running in the background as long as its main process remains active.

---

55. What is Docker Interactive Mode?
Definition: Interactive mode allows the user to interact directly with the process running inside a Docker container through the terminal.
Simple meaning:
Interactive mode allows us to type input and interact with the container process.
The -i option keeps standard input open.
The -t option allocates a terminal.
Together, -it are commonly used when we want an interactive shell.
Why do we use Interactive Mode?
We use interactive mode for:

- Opening a shell.
- Debugging.
- Testing commands.
- Working directly with a container terminal.

Interactive mode is especially useful when the application or shell requires user input.

---

56. What is Docker Port?
Definition: A Docker Port is a network port associated with an application or service running inside a Docker container.
Simple meaning:
A container port is the port on which an application inside the container listens for network connections.
For example, an application may listen on:
8080
inside the container.
Why do we use Container Ports?
We use container ports to:

- Allow applications to receive network traffic.
- Define how services communicate.
- Identify the network port used by an application.

A container port and a host port are not necessarily the same.
The container port is inside the container, while the host port belongs to the Docker host.

---

57. What is Docker Host?
Definition: A Docker Host is the system on which Docker Engine runs and where Docker containers are created and executed.
Simple meaning:
Docker Host is the machine that provides the environment for running Docker containers.
The Docker Host can be:

- A physical server.
- A virtual machine.
- A cloud server.
- A developer's computer.

The Docker Host provides resources such as:

- CPU.
- Memory.
- Storage.
- Networking.

Docker Engine runs on the Docker Host and manages the containers running there.
The basic relationship is:
Docker Host → Docker Engine → Docker Containers

---

58. What is Docker Runtime?
Definition: A Docker Runtime is the component responsible for starting and managing the execution of containers.
Simple meaning:
Container runtime is the software layer that actually creates and runs container processes.
Docker uses a container runtime to handle low-level container operations.
The runtime is responsible for tasks such as:

- Creating container processes.
- Setting up namespaces.
- Setting up control groups.

- Applying container isolation.
- Starting the container process.

Modern Docker environments commonly use containerd and an OCI-compatible runtime such as runc.
The basic relationship is:
Docker Engine → containerd → OCI Runtime → Container Process

---

59. What is Docker Namespace?
Definition: A Docker Namespace is a Linux kernel isolation mechanism used to isolate different aspects of a container's processes from other processes on the host.
Simple meaning:
Namespaces make a container's processes appear to have their own isolated environment.
Namespaces can isolate things such as:

- Process IDs.
- Network interfaces.
- Mount points.
- Hostnames.
- Users.
- Inter-process communication.

Why do we use Namespaces?
We use namespaces to provide process-level isolation between containers and the host system.
For example, processes inside one container are isolated from processes running in another container.
The main purpose is:
Namespace → Process and resource isolation

---

60. What is Docker Cgroups?
Definition: Docker Cgroups, or Control Groups, are Linux kernel mechanisms used to control and limit the resources available to container processes.
Simple meaning:
Cgroups control how much system resources a container can use.
Resources that can be controlled include:

- CPU.
- Memory.
- Block I/O.
- Other system resources supported by the environment.

Why do we use Cgroups?
We use Cgroups to:

- Limit resource usage.
- Prevent one container from consuming excessive resources.
- Allocate resources between workloads.
- Improve resource management.

The basic difference is:
Namespaces → Isolation
Cgroups → Resource control
Together, they are important building blocks for container isolation and resource management.